

Name: 徐靖甯

Date:09/26

Topic: Testing, Debugging, Exceptions, Assertions

1. Today's Challenge

Core concept from today's OCW lecture:

I learned how to use debugging techniques and exception handling to find and handle errors in a program. I also learned that testing normal cases and edge cases can help me check whether my program works correctly

AI-generated coding challenge title: Find All Clean Numbers

Challenge: Find All "Clean Numbers"

A positive integer is called **clean** if it is not divisible by any smaller clean number greater than 1.

For example:

- 2 is clean.
- 3 is clean.
- 4 is not clean because it is divisible by 2.
- 5 is clean.
- 6 is not clean because it is divisible by 2 and 3.
- 7 is clean.

Given an integer $n > 1$, return a list of all clean numbers less than or equal to n .

Input

A single integer n .

Output

Return a list containing all clean numbers from 2 through n , in ascending order.

Constraints

- $2 \leq n \leq 1000$
- The returned list must contain no duplicates.
- The numbers must appear in increasing order.

2. My Initial Approach — Before AI Help

Before asking AI for hints, briefly describe how you planned to solve the problem.

My approach:

I check every number from 2 to n and determine whether it could be divided evenly by any smaller number that had already been found. I would use a boolean variable to record whether the number had a divisor. If it had a divisor, I would not add it to the list. If it did not have a divisor, I would add it to the list. I would then continue checking until reaching n .

3. AI Tutor Help

Did you ask the AI Tutor for help?

☐ No — I solved it independently

☒ Yes — I received one or more hints

The most useful hint/question from AI was:

The AI asked me to explain why I would use a boolean flag and suggested thinking about what would happen when a number has more than one possible divisor.

It helped me realize that:

A boolean flag can keep track of whether a certain condition has already happened. In this problem, I can use it to remember whether the current number has been divided evenly by another number. This makes the decision clearer because I only need to check whether the flag is True or False after the loop.

4. My Revision

Did you change your approach or code after interacting with AI?

☐ No

☒ Yes

What did you change, and why?

I changed my approach by using a boolean flag to record whether the current number has a divisor. At first, I thought I could directly decide whether to add a number during each comparison. However, this could cause a problem because one number may need to be compared with several other numbers. Using a flag allows me to finish the comparisons first and then decide whether the number should be added to the list. This makes my logic easier to understand and debug.

5. Verification

My final program:

☒ Passed the provided examples

☒ Passed additional edge cases

☐ Still has unresolved problems

One edge case I tested:

Input: $n = 2$

Expected output: [2]

Actual output: [2]

6. One-Minute Reflection

What idea from the OCW lecture did you transfer to this new problem?

I transferred the idea of using a boolean flag from the lecture's prime-number example. Instead of immediately adding a number while checking it, I used the flag to record whether a condition was found and made the final decision after checking.

One thing I understand better now:

I understand better how a boolean flag can help control the logic of a program, especially when there are several conditions that need to be checked before making a final decision.

One thing I am still unsure about:

I am still a little unsure about when I should use exception handling and when I should use an assertion. I understand that both can detect problems, but I want to practice more to understand which situation is more appropriate for each one.